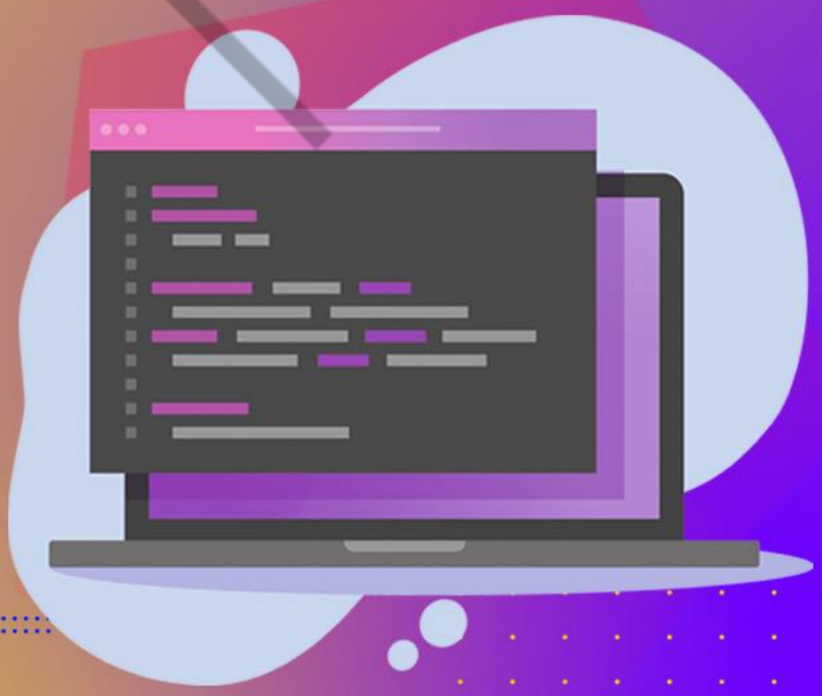
FIAP ON

SAFRA DO CONHECIMENTO CIBERNÉTICO - PRAGAS VIRTUAIS

DDL AVANÇADO: SEQUENCES E IDENTITY NO SQL



9

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Resultado da consulta utilizando o NEXTVAL 10
Figura 2 - Exemplo do “GAP”, quando a pseudocoluna é referenciada 10
Figura 3 - Resultado da consulta utilizando o CURRVAL. 11
Figura 4 - Estrutura da tabela DUAL do sistema Oracle Database 11
Figura 5 - Estrutura da SEQUENCE após alteração 13
Figura 6 - Modelo Relacional – Tabela CATEGORIA.....16
Figura 7 - Modelo Relacional – Tabela AUTOR 17

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de código-fonte SQL para criação de Sequence 5
Código-fonte 2 - Comentários sobre a estrutura da sequence (1).....7
Código-fonte 3 - Comentários sobre a estrutura da sequence (2).....7
Código-fonte 4 - Criação de tabela para uso com a SEQUENCE 8
Código-fonte 5 - Exemplo de criação de SEQUENCE 8
Código-fonte 6 - Criação de tabela, usando a sequence como valor default 9
Código-fonte 7 - Consultando a pseudocoluna NEXTVAL 10
Código-fonte 8 - Consultando a pseudocoluna CURRVAL 11
Código-fonte 9 - Preenchendo uma chave primária com SEQUENCE 12
Código-fonte 10 - Exemplo de alteração de SEQUENCE 13
Código-fonte 11 - Exemplo de exclusão de SEQUENCE.....14
Código-fonte 12 - Coluna IDENTITY criada com *GENERATED ALWAYS* 14
Código-fonte 13 - Coluna IDENTITY criada com *GENERATED BY DEFAULT* 15
Código-fonte 14 - Definição e uso do IDENTITY.....15
Código-fonte 15 - Criação e uso de SEQUENCE.....19
Código-fonte 16 - Criação e uso de IDENTITY 19

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 DDL AVANÇADO: SEQUENCES E IDENTITY NO SQL.....5
1.1 Sequências.....5
1.1.1 Criando uma sequência.....5
1.1.2 Exemplos de aplicações utilizando SEQUENCES 7
1.1.3 Sequência: Pseudocolunas CURRVAL e NEXTVAL.....9
1.1.4 Realizando a inserção de valores auto numéricos a partir da sequência.....11
1.1.5 Alterando uma sequência.....13
1.1.6 Excluindo uma sequência.....13
1.1.7 Identity no Oracle Database 14
1.1.8 Principais SGBDs que usam ou não sequência.....16
1.1.9 Hands ON – Aplicação de SEQUENCE e IDENTITY 16
1.1.9.1 MODELO RELACIONAL - Estudo de Caso: SLV – Sistema de Cadastro de Livros.....16
1.1.9.2 Instruções SQL – Comandos DDL – Para implementação SEQUENCE e IDENTITY 17
REFERÊNCIAS.....20
GLOSSÁRIO 21

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 DDL AVANÇADO: SEQUENCES E IDENTITY NO SQL

1.1 Sequências

Uma sequência é um item de banco de dados, cuja função é gerar uma série de números inteiros. A sequência é normalmente utilizada para preencher uma coluna chave primária numérica, com valores numéricos inteiros.

Ao utilizar uma sequência, os números são gerados de forma automática pelo SGBD (Sistema Gerenciador de Banco de Dados). Os números gerados pela sequência também são conhecidos por valores autonuméricos ou autonumeração.

Podemos gerar sequências crescentes ou decrescentes.

1.1.1 Criando uma sequência

Para criar uma sequência, devemos utilizar o comando CREATE SEQUENCE, que é um comando dentro da divisão DDL, da linguagem SQL.

Sintaxe básica do comando CREATE SEQUENCE:

```
CREATE SEQUENCE nome_sequencia
[ START WITH num_inicio ]
[ INCREMENT BY  num_incremento ]
[ { MAXVALUE  num_maximo | NOMAXVALUE } ]
[ { MINVALUE  num_minimo | NOMINVALUE } ]
[ { CYCLE | NOCYCLE} ]
[ { CACHE   num_cache | NOCACHE } ]
[ { ORDER | NOORDER } ];
```

Código-fonte 1 - Exemplo de código-fonte SQL para criação de Sequence
Fonte: Oracle Database (2009)

Onde:

- ***nom_sequencia:*** nome da sequência.
- ***num_inicio:*** número inteiro usado para iniciar a sequência. O número inicial padrão é 1.
- ***num_incremento:*** número inteiro para incrementar a sequência. O número de incremento padrão é 1.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- ***num_maximo:*** número inteiro máximo da sequência. Deve ser maior ou igual ao número inicial. Deve ser maior que o número mínimo.
- **NOMAXVALUE:** especifica que o máximo é 10^{27} para uma sequência crescente ou -1 para uma sequência decrescente. A opção NOMAXVALUE é padrão.
- ***num_minimo*:** número inteiro mínimo da sequência. Deve ser menor ou igual ao número inicial da sequência. Deve ser menor que o número máximo da sequência.
- **NOMINVALUE:** especifica que o mínimo é 1 para uma sequência crescente ou 10^{-26} para uma sequência decrescente. NOMINVALUE é o padrão.
- **CYCLE:** significa que a sequência gera números inteiros mesmo depois de atingir seu valor máximo ou mínimo. Quando uma sequência crescente atinge seu valor máximo, o próximo valor gerado é o mínimo. Quando uma sequência decrescente atinge seu valor mínimo, o próximo valor gerado é o máximo.
- **NOCYCLE:** significa que a sequência não pode gerar mais número inteiro algum após atingir seu valor máximo ou mínimo. NOCYCLE é o padrão.
- ***num_cache*:** é o número de valores inteiros a manter na memória. O número de valores inteiros padrão colocados na cache é 20. O número mínimo de valores inteiros que podem ser colocados no cache é 2. O número máximo de valores inteiros que podem ser colocados no cache é determinado pela fórmula *CEIL (num_maximo – num_minimo) / ABS (num_incremento)*.
- **NOCACHE:** significa sem cache. Isso impede o banco de dados de alocar valores previamente para a sequência, o que evita lacunas numéricas na sequência, mas reduz o desempenho. As lacunas ocorrem porque os valores alocados no cache são perdidos quando um banco de dados é fechado. Se for omitida CACHE/NOCACHE, o banco de dados colocará 20 números de sequência na CACHE, por padrão.
- **ORDER:** garante que os números inteiros sejam gerados na ordem da solicitação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **NOORDER:** não garante que os números inteiros sejam gerados na ordem de solicitação. NOORDER é o padrão.

Exemplos e comentários sobre a estrutura de uma SEQUENCE:

```
-- COMENTÁRIOS SOBRE A ESTRUTURA DA SEQUENCE
CREATE SEQUENCE <nome_da_sequencia>
     START WITH 1  -- Primeiro número a ser gerado
   INCREMENT BY 1  -- Valor do incremento (≠ zero)
     NOMAXVALUE    -- Sem definição de limite máximo
          CACHE 50 -- O default é "CACHE 20"
        NOCYCLE ;  -- Após chegar ao limite, não reinicia
```

Código-fonte 2 - Comentários sobre a estrutura da sequence (1)
Fonte: Elaborado pelo autor (2022)

```
-- COMENTÁRIOS SOBRE A ESTRUTURA DA SEQUENCE
CREATE SEQUENCE <nome_da_sequencia>
   INCREMENT BY 1    -- Primeiro número a ser gerado
     START WITH 1    -- Valor do incremento (≠ zero)
       MAXVALUE 999 -- Valor máximo da sequência
        NOCACHE      -- sem cache
          CYCLE; --Após chegar ao limite, reinicia a sequência
```

Código-fonte 3 - Comentários sobre a estrutura da sequence (2)
Fonte: Elaborado pelo autor (2022)

Importante: as Lacunas/Intervalos “Gaps” de valores da sequência podem ocorrer quando: ocorrer um rollback (uma transação é desfeita); falha no sistema; e quando uma sequência é usada por duas ou mais tabelas.

1.1.2 Exemplos de aplicações utilizando SEQUENCES

Para exemplificar aplicações utilizando SEQUENCES, iremos construir (implementar) a tabela a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Apresentamos o código na linguagem SQL, que permite criar a tabela, para que possamos trabalhar outras aplicações utilizando SEQUENCE, a partir da sintaxe geral que foi apresentada e seguindo as recomendações para implementação do código.

```
-- CRIAÇÃO DE TABELA PARA EXEMPLIFICAR SEQUENCES

-- APAGAR A TABELA PARA TESTES
DROP TABLE T_TESTE_DDL_SEQUENCE;

-- CRIAR UMA TABELA PARA TESTES
CREATE TABLE T_TESTE_DDL_SEQUENCE
(
    CD_TESTE_SQ     NUMBER(3)        NOT NULL,
    NM_TESTE_SQ     VARCHAR2(30)    NOT NULL
);
-- ADICIONAR A CONSTRAINT CHAVE PRIMARIA
   ALTER TABLE T_TESTE_DDL_SEQUENCE
ADD CONSTRAINT PK_TESTE_DDL_SEQUENCE
   PRIMARY KEY (CD_TESTE_SQ);
```

Código-fonte 4 - Criação de tabela para uso com a SEQUENCE
Fonte: Elaborado pelo autor (2022)

Exemplo do comando DDL, para criar a sequência de números inteiros que será utilizado na chave primária para a tabela de testes, descrita anteriormente:

```
-- CRIAR A SEQUÊNCIA PARA GERAR UM CODIGO AUTO NUMERICO

CREATE SEQUENCE Q_TESTE_DDL_SEQUENCE
   INCREMENT BY 1
  START WITH 1
    MAXVALUE 999
     NOCACHE
     NOCYCLE;
```

Código-fonte 5 - Exemplo de criação de SEQUENCE
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Poderíamos criar a tabela para os testes da SEQUENCE utilizando a sequência como valor DEFAULT. Dessa maneira, a cada inserção, um novo valor é gerado para a chave primária dessa tabela.

Nesse caso, devemos criar a sequência e depois associá-la ao comando de criação da tabela como valor default.

Apresentamos o comando DDL, que permite realizar essa implementação.

```
-- CRIAÇÃO DE TABELA PARA EXEMPLIFICAR SEQUENCES

-- APAGAR A TABELA PARA TESTES
DROP TABLE T_TESTE_DDL_SEQUENCE;

-- CRIAR UMA TABELA PARA TESTES
CREATE TABLE T_TESTE_DDL_SEQUENCE
(
    CD_TESTE_SQ NUMBER(3)
        DEFAULT SQ_TESTE_DDL_SEQUENCE.NEXTVAL NOT NULL,
    NM_TESTE_SQ     VARCHAR2(30)    NOT NULL
);
-- ADICIONAR A CONSTRAINT CHAVE PRIMARIA
   ALTER TABLE T_TESTE_DDL_SEQUENCE
ADD CONSTRAINT PK_TESTE_DDL_SEQUENCE
   PRIMARY KEY (CD_TESTE_SQ);
```

Código-fonte 6 - Criação de tabela, usando a sequence como valor default
Fonte: Elaborado pelo autor (2022)

1.1.3 Sequência: Pseudocolunas CURRVAL e NEXTVAL

A sequência gera uma série de valores numéricos inteiros.

Pseudocolunas CURRVAL e NEXTVAL:

Podemos utilizar as duas pseudocolunas contidas nas sequências para saber o valor atual da sequência e o próximo valor da sequência.

Estas pseudocolunas são chamadas de CURRVAL e NEXTVAL.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Quando utilizamos o NEXTVAL para determinar o próximo valor da sequência, o próximo valor é alterado, causando um “GAP” de valores, dependendo de como for utilizado.

Exemplo da consulta a pseudocoluna NEXTVAL, da sequência “SQ_TESTE_DDL_SEQUENCE”:

```
-- CONSULTANDO A PSEUDOCOLUNA NEXTVAL

      SELECT SQ_TESTE_DDL_SEQUENCE.NEXTVAL FROM DUAL;
```

Código-fonte 7 - Consultando a pseudocoluna NEXTVAL
Fonte: Elaborado pelo autor (2022)

Resultado da consulta:

NEXTVAL
2

Figura 1 - Resultado da consulta utilizando o NEXTVAL
Fonte: Elaborado pelo autor (2022)

Exemplo do “GAP”, quando a pseudocoluna é referenciada.

	CD_TESTE_SQ	NM_TESTE_SQ
1	2	TESTE
2	3	TESTE
3	4	TESTE
4	5	TESTE
5	8	TESTE

Figura 2 - Exemplo do “GAP”, quando a pseudocoluna é referenciada
Fonte: Elaborado pelo autor (2022)

Nesta figura, é possível observar várias linhas inseridas, entre uma e outra inserção, e recuperar o NEXTVAL que causou o “GAP”. Note que a primeira linha faz referência ao código igual a 2, e a última linha (linha 5) faz referência ao código igual a 8. Portanto, temos um “GAP” no início (primeira linha da tabela) e antes da última linha (entre as linhas 4 e 5), observamos que faltam os códigos de número 6 e 7, o “GAP” foi provocado, pois executamos várias vezes o NEXTVAL, como apresentado no código-fonte “Criação de tabela, usando a sequence como valor default”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Quando utilizamos o CURRVAL para determinar o valor atual da sequência, não há alteração no próximo valor.

Importante: caso a sequência não tenha sido iniciada, ocorrerá um erro se CURRVAL for utilizado. Para isso, inicie a sequência utilizando NEXTVAL.

Exemplo da consulta a pseudocoluna CURRVAL, da sequência "SQ_TESTE_DDL_SEQUENCE":

```
-- CONSULTANDO A PSEUDOCOLUNA CURRVAL

      SELECT SQ_TESTE_DDL_SEQUENCE.CURRVAL FROM DUAL;
```

Código-fonte 8 - Consultando a pseudocoluna CURRVAL
Fonte: Elaborado pelo autor (2022)

Resultado da consulta:

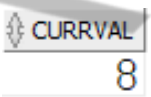


Figura 3 - Resultado da consulta utilizando o CURRVAL.
Fonte: Elaborado pelo autor (2022)

A tabela **DUAL** faz parte do sistema ORACLE DATABASE, pertence ao usuário SYS, utilizado por desenvolvedores, DBA e pela própria ORACLE, quando é necessário realizar uma consulta (operação SELECT), onde não é necessário extrair dados de uma tabela. Essa tabela não pode ser excluída.

Possui apenas uma coluna, nomeada "DUMMY" do tipo VARCHAR2(1) com o conteúdo "X".

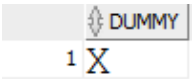


Figura 4 - Estrutura da tabela DUAL do sistema Oracle Database
Fonte: Elaborado pelo autor (2022)

1.1.4 Realizando a inserção de valores auto numéricos a partir da sequência

Normalmente, utilizamos a sequência para preencher uma coluna de chave primária, com valores auto numéricos de forma automática.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Atenção: quando utilizamos a sequência para preencher uma coluna de chave primária, recomenda-se usar NOCACHE para evitar "GAPS" na sequência. Vale ressaltar que utilizando NOCACHE, há uma redução de desempenho. Se as lacunas "GAPS" não forem um problema, utilize CACHE.

Preenchendo uma chave primária com uma sequência

A seguir, apresentamos o comando DML que permite realizar a inserção de linhas (registros) na tabela usada como exemplo.

Exemplo de preenchimento da chave primária da tabela "T_TESTE_DDL_SEQUENCE" utilizando a sequência "**SQ_TESTE_DDL_SEQUENCE**".

Utilizaremos o comando INSERT, que será abordado mais adiante.

```
-- INSERÇÃO DE LINHA UTILIZANDO VALORES AUTO NUMÉRICOS DE
-- FORMA AUTOMÁTICA
INSERT INTO T_TESTE_DDL_SEQUENCE
     VALUES (SQ_TESTE_DDL_SEQUENCE.NEXTVAL, 'TESTE');

-- INSERÇÃO DE LINHA UTILIZANDO VALORES AUTO NUMÉRICOS
-- DE FORMA AUTOMÁTICA
-- QUANDO A SEQUÊNCIA É DEFINIDA COMO VALOR DEFAULT PARA O
-- CAMPO CHAVE PRIMÁRIA
INSERT INTO T_TESTE_DDL_SEQUENCE (NM_TESTE_SQ)
     VALUES ('TESTE');
```

Código-fonte 9 - Preenchendo uma chave primária com SEQUENCE
Fonte: Elaborado pelo autor (2022)

Quando utilizamos a sequência como valor default para a chave primária, no momento da inserção podemos omitir o campo chave primária da declaração do comando. Isto é, ao criarmos a instrução dessa forma, temos que **obrigatoriamente** informar quais colunas e respectivos valores serão inseridos.

Importante: esse é o comando que será utilizado no momento da persistência de dados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.5 Alterando uma sequência

Podemos alterar uma sequência utilizando a instrução ALTER SEQUENCE. Existem algumas limitações que devemos considerar:

- O valor inicial de uma sequência não pode ser alterado.
- O valor mínimo não pode ser maior que o valor atual da sequência.
- O valor máximo não pode ser menor do que o valor atual da sequência.
- Para alterar uma sequência é necessário ser proprietário dela.
- Somente os números futuros da sequência são afetados após a alteração.
- A sequência deve ser eliminada e recriada para que seja reiniciada em um número diferente.

Exemplo do comando DDL, para alterar uma sequência de números inteiros:

```
-- ALTERANDO UMA SEQUÊNCIA

      ALTER SEQUENCE SQ_TESTE_DDL_SEQUENCE
        INCREMENT BY 5
            MAXVALUE 900;
```

Código-fonte 10 - Exemplo de alteração de SEQUENCE
Fonte: Elaborado pelo autor (2022)

Visualização da estrutura da sequência, após a execução do comando DDL de alteração.

SEQUENCE_NAME	INCREMENT_BY	MAX_VALUE	LAST_NUMBER
SQ_TESTE_DDL_SEQUENCE	1	999	9

Figura 5 - Estrutura da SEQUENCE após alteração
Fonte: Elaborado pelo autor (2022)

1.1.6 Excluindo uma sequência

Podemos excluir uma sequência utilizando a instrução DROP SEQUENCE.

Exemplo de comando DDL, para excluir uma sequência:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- EXCLUINDO UMA SEQUÊNCIA

      DROP SEQUENCE SQ_TESTE_DDL_SEQUENCE;
```

Código-fonte 11 - Exemplo de exclusão de SEQUENCE
Fonte: Elaborado pelo Autor (2022)

1.1.7 Identity no Oracle Database

Essa feature (funcionalidade) foi adicionada na versão 12c do Oracle Database, antes disso era possível gerar valores auto numéricos de forma automática, através da SEQUENCE.

O IDENTITY é vinculado a coluna chave primária de uma tabela, desta forma são atribuídos valores numéricos inteiros de forma automática. Esses valores são gerados a partir de uma sequência, que é gerenciada pelo próprio SGBD.

O Oracle Database possui um gerador de sequência (SG), que é um serviço que gera uma sequência de números inteiros. Cada coluna IDENTITY vinculada a uma tabela requer seu próprio gerador de sequência. O gerador de sequência é responsável por obter e fornecer valores para a coluna IDENTITY.

A coluna IDENTITY deve ser criada como parte do comando CREATE TABLE.

Podemos criar a coluna IDENTITY de duas maneiras:

- **Forma 1:** coluna IDENTITY criada com GENERATED ALWAYS.

```
-- COLUNA IDENTITY CRIADA COM GENERATE ALWAYS

      CREATE TABLE T1 (
          id INTEGER GENERATED ALWAYS AS IDENTITY
          (START WITH 2 INCREMENT BY 2 MAXVALUE 200 NO CYCLE),
          name STRING,
          PRIMARY KEY (id)
      );
```

Código-fonte 12 - Coluna IDENTITY criada com *GENERATED ALWAYS*
Fonte: Oracle (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Forma 2:** coluna IDENTITY criada com GENERATED BY DEFAULT.

```
-- COLUNA IDENTITY CRIADA COM GENERATE BY DEFAULT

      CREATE TABLE T2 (
          id LONG GENERATED BY DEFAULT AS IDENTITY
          (START WITH 1 INCREMENT BY 1 CYCLE CACHE 200),
          account_id INTEGER,
          name STRING,
          PRIMARY KEY (account_id)
      );
```

Código-fonte 13 - Coluna IDENTITY criada com *GENERATED BY DEFAULT*
Fonte: Oracle (2022)

Apresentamos o código na linguagem SQL que permite criar a tabela para que possamos trabalhar aplicações utilizando IDENTITY a partir da sintaxe geral que foi apresentada e seguindo as recomendações para implementação do código.

```
-- EXEMPLO COMPLETO DE DEFINIÇÃO E USO DO IDENTITY

-- EXCLUSÃO DA TABELA EXEMPLO
DROP TABLE T_TESTE_IDENTITY;

-- CRIAÇÃO DA TABELA INDICANDO A COLUNA IDENTITY
CREATE TABLE T_TESTE_IDENTITY (
  ID_TESTE          NUMBER(5) GENERATED ALWAYS AS IDENTITY,
  DESC_TESTE        VARCHAR2(30)
);
-- DEFINIÇÃO DA CHAVE PRIMÁRIA
   ALTER TABLE T_TESTE_IDENTITY
ADD CONSTRAINT PK_TESTE_IDENTITY
   PRIMARY KEY (ID_TESTE);
-- INSERÇÃO A PARTIR DA COLUNA IDENTITY
INSERT INTO T_TESTE_IDENTITY (DESC_TESTE) VALUES ('TESTE ID');
-- CONSULTA AOS DADOS DA TABELA DE EXEMPLO
SELECT * FROM T_TESTE_IDENTITY;
```

Código-fonte 14 - Definição e uso do IDENTITY
Fonte: Elaborado pelo Autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.1.8 Principais SGBDs que usam ou não sequência

Os SGBDs permitem a criação de auto numéricos, mas observamos que cada fabricante implementa essa funcionalidade de formas diferentes. Exemplificando, SGBDs como Oracle, DB2 e PostGreSQL utilizam sequências para gerar valores numéricos inteiros.

No SQL Server podemos qualificar uma coluna com o IDENTITY (valor_inicial, incremento). O IDENTITY é vinculado a uma coluna da tabela, que permite gerar os valores numéricos inteiros de forma automática.

No MySQL podemos qualificar uma coluna com o AUTO_INCREMENT. Que é vinculado a uma coluna da tabela, que permite gerar os valores numéricos inteiros de forma automática.

1.1.9 Hands ON – Aplicação de SEQUENCE e IDENTITY

1.1.9.1 MODELO RELACIONAL - Estudo de Caso: SLV – Sistema de Cadastro de Livros

A partir do fragmento do Modelo Relacional a seguir, podemos realizar a leitura e entendimento das tabelas e criar as instruções SQL e comandos DDL para implementar SEQUENCE e IDENTITY.

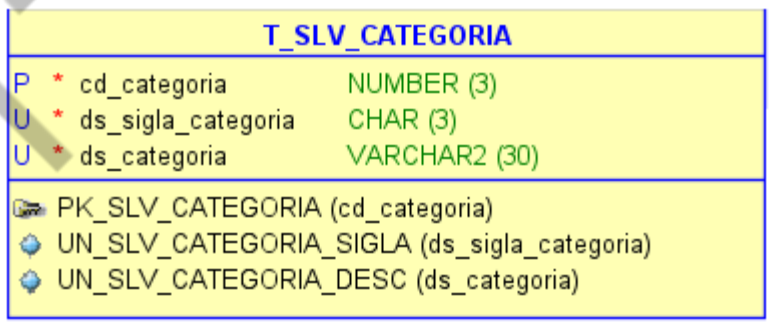


Figura 6 - Modelo Relacional – Tabela CATEGORIA
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

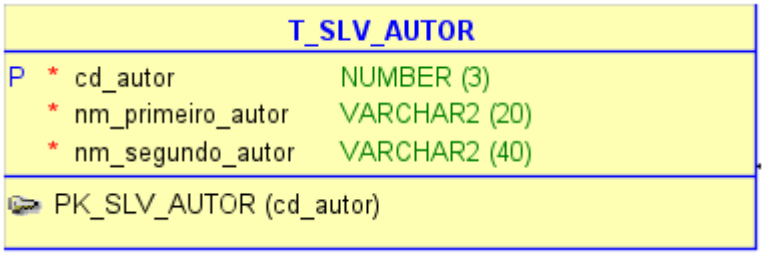


Figura 7 - Modelo Relacional – Tabela AUTOR
Fonte: Elaborado pelo autor (2022)

1.1.9.2 Instruções SQL – Comandos DDL – Para implementação SEQUENCE e IDENTITY

Esta atividade é baseada nos conceitos abordados nas seções anteriores. E será composta por um conjunto de instruções, comandos DDL, que permite implementar SEQUENCE e IDENTITY, através do SGBD Oracle Database.

Os comandos devem ser inseridos em um arquivo e executados utilizando-se de uma ferramenta que interprete e execute os comandos.

Como sugestão, podemos utilizar a ferramenta SQL Developer da Oracle.

O script (arquivo com comandos SQL) deve possuir um nome que ajude na sua identificação, por exemplo: “SCRIPT_DDL_SEQUENCE_IDENTITY.SQL”, com a extensão SQL.

A primeira aplicação será a geração da SEQUENCE e uso na tabela CATEGORIA:

```
-- EXCLUIR A TABELA CASO EXISTA
DROP TABLE t_slv_categoria  CASCADE CONSTRAINTS;

-- EXCLUIR A SEQUENCIA CASO EXISTA
DROP SEQUENCE SQ_SLV_CATEGORIA;

-- CRIAR A SEQUENCIA QUE SERA USADA NA CATEGORIA
CREATE SEQUENCE SQ_SLV_CATEGORIA
     START WITH 1
   INCREMENT BY 1
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        MAXVALUE 999
         NOCACHE
         NOCYCLE;

-- CRIAR A TABELA CATEGORIA, COM A SEQUENCIA COMO
-- DEFAULT PARA A CHAVE PRIMÁRIA
-- CRIAÇÃO DA TABELA CATEGORIA
CREATE TABLE t_slv_categoria (
    cd_categoria        NUMBER(3)        DEFAULT
                        SQ_SLV_CATEGORIA.NEXTVAL NOT NULL,
    ds_sigla_categoria  CHAR(3)          NOT NULL,
    ds_categoria        VARCHAR2(30)     NOT NULL
);
-- ADIÇÃO DA CONSTRAINT CHAVE PRIMARIA
   ALTER TABLE t_slv_categoria
ADD CONSTRAINT pk_slv_categoria
   PRIMARY KEY ( cd_categoria );
-- ADIÇÃO DA CONSTRAINT UNIQUE
   ALTER TABLE t_slv_categoria
ADD CONSTRAINT un_slv_categoria_sigla
        UNIQUE ( ds_sigla_categoria );
-- ADIÇÃO DA CONSTRAINT UNIQUE
   ALTER TABLE t_slv_categoria
ADD CONSTRAINT un_slv_categoria_desc
        UNIQUE ( ds_categoria );

-- INSERINDO VALORES NA TABELA, UTILIZANDO A SEQUENCIA
INSERT INTO T_SLV_CATEGORIA (DS_SIGLA_CATEGORIA, DS_CATEGORIA)
VALUES('AA', 'CATEGORIA A');
INSERT INTO T_SLV_CATEGORIA (DS_SIGLA_CATEGORIA, DS_CATEGORIA)
VALUES('BB', 'CATEGORIA B');
INSERT INTO T_SLV_CATEGORIA (DS_SIGLA_CATEGORIA, DS_CATEGORIA)
VALUES('CC', 'CATEGORIA C');
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
-- VISUALIZANDO AS LINHAS INSERIDAS
SELECT * FROM T_SLV_CATEGORIA;
```

Código-fonte 15 - Criação e uso de SEQUENCE
Fonte: Elaborado pelo Autor (2022)

A segunda aplicação será a geração da coluna IDENTITY e uso na tabela AUTOR:

```
-- EXCLUIR A TABELA CASO EXISTA
DROP TABLE t_slv_autor CASCADE CONSTRAINTS;

-- CRIAÇÃO DA TABELA AUTOR
CREATE TABLE t_slv_autor (
    cd_autor            NUMBER(3)
                        GENERATED ALWAYS AS IDENTITY,
    nm_primeiro_autor   VARCHAR2(20) NOT NULL,
    nm_segundo_autor    VARCHAR2(40) NOT NULL
);
-- ADIÇÃO DA CONSTRAINT CHAVE PRIMARIA
   ALTER TABLE t_slv_autor
ADD CONSTRAINT pk_slv_autor
   PRIMARY KEY ( cd_autor );

-- INSERINDO VALORES NA TABELA, UTILIZANDO IDENTITY
INSERT INTO T_SLV_AUTOR (NM_PRIMEIRO_AUTOR, NM_SEGUNDO_AUTOR)
     VALUES ('AUTOR', 'SOBRENOME AUTOR');
INSERT INTO T_SLV_AUTOR (NM_PRIMEIRO_AUTOR, NM_SEGUNDO_AUTOR)
     VALUES ('MARIA','DE ALBUQUERQUE');
INSERT INTO T_SLV_AUTOR (NM_PRIMEIRO_AUTOR, NM_SEGUNDO_AUTOR)
     VALUES ('MARIO','DE ANDRADE');

-- VISUALIZANDO AS LINHAS INSERIDAS
SELECT * FROM T_SLV_AUTOR;
```

Código-fonte 16 - Criação e uso de IDENTITY
Fonte: Elaborado pelo Autor (2022)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S.B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

HEUSER, C. A. **Projeto de Banco de Dados. Série Livros Didáticos.** v. 4. 6. ed. Rio Grande do Sul: Bookman, 2009.

MACHADO, Felipe Nery R. **Banco de Dados - Projeto e Implementação**. 2. ed. São Paulo: Érica, 2005.

PRICE, J. **Oracle Database 11g SQL**. São Paulo: Bookman, 2009.

PUGA, S.; FRANÇA, E.; GOYA, M. **Banco de Dados: Implementação em SQL, PL/SQL e Oracle 11g.** São Paulo: Pearson Education do Brasil, 2013.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 5. ed. Rio de Janeiro: Campus, 2006.

SQL Reference Guide. Using the IDENTITY Column. **Oracle**. Disponível em: <https://docs.oracle.com/en/database/other-databases/nosql-database/22.1/sqlreferencefornosql/identity-column.html>. Acesso em: 24 set. 2024.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

INSERT	Comando da linguagem SQL responsável por inserir registros ou linhas nas tabelas.
DUAL	É uma tabela do sistema ORACLE, pertence ao usuário SYS, utilizado por desenvolvedores, DBA e pela própria ORACLE, quando é necessário realizar uma consulta (operação SELECT), onde não é necessário extrair dados de uma tabela. Não pode ser excluída. Possui apenas uma coluna, nomeada “DUMMY” do tipo VARCHAR2(1) com o conteúdo “X”.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com